

LeetCode 3070

Count Submatrices with Top-Left Element and Sum Less Than k

Author: Arpit Raj **Institution:** LNMIIT Jaipur **Date:** July 9, 2026

Targeted at: Barclays

Problem Analysis & Approach

The core of this problem relies on computing the sum of submatrices efficiently. Instead of calculating the sum for each submatrix individually, we can optimize the approach using a **2D Prefix Sum Array**.



Important Observations (Read Carefully!)

- The problem strictly states: *Count submatrices with top-left element $(0,0)$ whose sum is $\leq k$.*
- **Notice:** It does **not** say "count all possible submatrices".
- Because the top-left corner is permanently fixed at $(0,0)$, our search space is significantly reduced. We only need to check the pre-computed prefix sum for the bottom-right corner $(r2, c2)$.
- If $\text{pref}[r2][c2] \leq k$, we have found a valid submatrix!

C++ Implementation

Below is the accepted optimal implementation utilizing a 2D prefix sum technique to achieve $O(N \times M)$ time complexity.

```

class Solution {
public:
    int countSubmatrices(vector<vector<int>>& grid, int k) {

        int ans = 0;
        int n = grid.size();
        int m = grid[0].size();
        vector<vector<int>> pref(n, vector<int>(m));

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                pref[i][j] = grid[i][j];

                if(i > 0){
                    pref[i][j] += pref[i-1][j];
                }

                if(j > 0){
                    pref[i][j] += pref[i][j-1];
                }

                if(i > 0 && j > 0){
                    pref[i][j] -= pref[i-1][j-1];
                }
            }
        }

        for(int roff = 0; roff < n; roff++) {
            for(int coff = 0; coff < m; coff++) {
                // int r1 = 0;
                // int c1 = 0; // fix kar rkha he
                int r2 = roff;
                int c2 = coff;

                if(pref[r2][c2] <= k) ans++;
            }
        }

        return ans;
    }
};

```

